Mini Project 01 - Ballistic Flight Simulator v1.0

Python Engineering - Consolidating logic, loops, validation and Explicit Euler

This mini project closes the first block of the course. The goal is not to add lots of new syntax, but to integrate what you have learned into one program: variables, validation, conditionals, loops, state, accumulators, Explicit Euler and validation against an analytical solution.

1. Project objective

• Build a 1D vertical simulator with constant gravity and no aerodynamic drag.

• Propagate the state **[t, h, v]** using Explicit Euler with a fixed time step **dt**.

• Detect the approximate apogee and the first step that crosses the ground.

• Compare flight time, maximum altitude and final velocity with the analytical solution.

• Study how the error changes when **dt** is reduced.

• Deliver a clear, documented and reproducible script that can evolve into later versions.

2. Physical model

We use the same model as EX08. The positive vertical axis points upward and **g > 0** is the magnitude of gravity.

$$dh/dt = v$$

$$dv/dt = -g$$

$$h_{(k+1)} = h_k + v_k \, dt$$

$$v_{(k+1)} = v_k - g \, dt$$

$$t_{(k+1)} = t_k + dt$$

Key idea: compute the complete new state using values from state k, then update. This preserves Explicit Euler.

3. Analytical reference solution

The analytical solution does not generate the numerical trajectory; it is only an independent reference used to validate the simulator.

$$h_{max} = h_0 + v_0^2 / (2g)$$

$$t_f = (v_0 + \sqrt{v_0^2 + 2 g h_0}) / g$$

$$v_f = -\sqrt{v_0^2 + 2 g h_0}$$

4. Logical architecture

```
INPUTS
  h0, v0, t0, g, dt
       |
VALIDATION
       |
INITIAL STATE
  t=t0, h=h0, v=v0
       |
WHILE h >= 0
  1. compute h_new, v_new, t_new
  2. update max_h
  3. replace old state
  4. count iteration
       |
FINAL NUMERICAL RESULTS
       |
ANALYTICAL REFERENCE
       |
ERRORS + SUMMARY
```

5. Minimum requirements

Block	Requirement
Inputs	h0 >= 0, v0 > 0, g > 0, dt > 0; t0 may be any time reference.
Integrator	Explicit Euler. Do not use SciPy, RK4 or the closed-form trajectory to propagate the motion.
State	Use separate variables for current state and new state.
Maximum	Store max_h by comparing the current value with the accumulated maximum.
Stop	Finish after detecting the first state with h < 0.
Validation	Compute analytical references and percentage errors.
Output	Print a readable final summary with dt, iterations, h_max, t_f and v_f.

6. Important correction for velocity error

> **Because the analytical impact velocity is negative, the denominator of the relative error must use its absolute value: abs(analytical_v_f). If you divide directly by analytical_v_f, the percentage may become negative even though you are measuring an error magnitude.**

error_v_f = abs(analytical_v_f - v_f) / abs(analytical_v_f) * 100

7. Convergence experiment

Run the same simulator without changing the physical conditions and modify only **dt**. Record the results in a table.

dt [s]	h_max Euler [m]	Error h_max [%]	t_f Euler [s]	Error t_f [%]
1.0				
0.5				
0.1				
0.05				
0.01				

Main question: what do you observe as dt decreases? Do not only look for a small error; look at the trend.

8. Mini-project deliverables

```
Python_Engineering/
■■■ 01_python_bootcamp/
    ■■■ week_01_logic/
        ■■■ class_01_equation_to_algorithm/
            ■■■ mini_project/
                ■■■ ballistic_flight_simulator_v1/
                    ■■■ ballistic_flight_simulator_v1.py
                    ■■■ README.md
                    ■■■ notes/
```

• **ballistic_flight_simulator_v1.py**: the final program.

• **README.md**: short description of the model, inputs, outputs, equations and limitations.

• **notes/**: the EX08/Euler PDFs and this mini-project brief.

9. Test cases

Case	h0	v0	g	dt	Check
Baseline	0	50	9.81	0.1	Nominal run and analytical validation.
Large step	0	50	9.81	1.0	Clearly visible Euler error.
Small step	0	50	9.81	0.01	Closer agreement with reference.
Initial height	100	50	9.81	0.1	Use general analytical formulas.
Invalid	0	50	9.81	0	Program must stop before the while loop.

10. What NOT to do yet

• Do not use **solve_ivp**, RK2 or RK4.

• Do not convert the project to classes yet.

• Do not add drag, atmosphere, variable mass or thrust yet.

• Do not obsess over the exact impact instant yet; crossing between time steps will be studied later.

11. Optional bonus

> **If you want one extra challenge, also compute the approximate mechanical energy E = g h + v^2/2 per unit mass at each step and observe whether Euler preserves it exactly. This is not required to close the project.**

12. Completion criteria

Criterion	Done
Consistent inputs and validation	■
Explicit Euler correctly implemented	■
max_h, t_f and v_f computed	■
Correct analytical references	■
Correct percentage errors	■

Criterion	Done
Study with multiple dt values completed	■
Readable code and README	■

13. Learning Tracker

Concept	Status
Conditionals and validation	Consolidated
for / while	Consolidated
Dynamic state k -> k+1	Consolidated
Explicit Euler	Applied
Discretization error	Applied
Analytical validation	Applied
Convergence as dt decreases	To study in the project
Incremental simulator design	Introduced

Final goal: be able to explain why the simulator works, where its error comes from, and how to verify that it improves as dt is reduced.